IMPLEMENTATION NOTE • MHD-WAAM THESIS • PHASE 3

The Arc Heat Source

How Goldak's double ellipsoid is implemented in laserbeamFoam, and the three things the bare formula gets wrong

Sabbir Dipto | *branch* `waam-mhd` | *four files changed, one added* | *Zhao et al. (2021) Eq. 27*

THE ONE-PARAGRAPH VERSION

laserbeamFoam heats with a ray-traced laser. GMAW-WAAM heats with a moving arc, which the source paper models as Goldak's double ellipsoid — narrow in front, wide behind, because the torch is travelling. The obvious route, a `codedFvOption`, is unavailable: `TEqn.H` never calls `fvOptions`, so a coded source would be **silently ignored** — the case would run, the arc would do nothing, and nothing would say why. The source therefore enters the energy equation directly, as a peer of the laser term, following a conditional-source pattern the solver already uses twice.

1. The physics being implemented

Zhao Eq. 27. Two half-ellipsoids sharing a centre that travels with the torch, with different lengths fore and aft because a moving arc trails heat behind it:

```
q(x,y,z) = 6 sqrt(3) f Q / (a b c pi sqrt(pi))
          * exp( -3 [ (x-x0-Vt)^2/a^2 + (y-y0)^2/b^2 + (z-z0)^2/c^2 ] )

  a = a_f, f = f_f ahead of the torch
  a = a_r, f = f_r behind it
```

SYMBOL	MEANING	VALUE	SOURCE
Q	arc power into the pool, eta·U·I	1512 W	caseParameters
a_f, a_r	front and rear semi-axes	3 mm, 6 mm	Zhao, CALIBRATED
b, c	half-width and penetration depth	4 mm, 4 mm	Zhao, CALIBRATED
f_f, f_r	front/rear power split, sum = 2	0.6667, 1.3333	derived, $2a/(a_f+a_r)$
V	travel speed	0.02 m/s	caseParameters
x_0,y_0,z_0	arc start point	(0, 0, 0)	caseParameters

*The four lengths are **fitted, not physical**. Zhao obtained them by matching a measured first-layer pool, so they encode that rig, mesh and property set. They must be recalibrated.*

2. Why it is a solver patch and not a dictionary entry

The plan had assumed a `codedFvOption` — a snippet in a case dictionary, no solver changes. Reading the source killed that:

```
$ grep -n 'fvOptions' TEqn.H
   (no matches)

$ grep -n 'fvOptions' UEqn.H
  71:    fvOptions(rho, U)
  76: fvOptions.constrain(UEqn);
  97: fvOptions.correct(U);
```

The solver is asymmetric: the **momentum** equation accepts coded options, the **energy** equation does not. So arc pressure and plasma drag can stay dictionary snippets, while the heat source cannot.

> **THE FAILURE MODE THIS AVOIDS**
>
> A `codedFvOption` on an equation that never calls `fvOptions` does not error. It compiles, loads, and is never applied. The case runs to completion with no arc and no warning — and the missing heat would look like a calibration problem for as long as it took to find.
>
> A source written into the solver is visible in a diff and reviewable. That is the better outcome even setting the silent failure aside.

3. Where the code lives

All four solver files sit in one directory:

```
~/thesis/src/lbf-com/applications/solvers/laserbeamFoam/
```

FILE	ROLE	SIZE
`createFields.H`	Reads `caseParameters`, derives f_f/f_r, resolves the symmetry factor, declares the `arcSource` field, prints a summary	+62 lines
`updateArcSource.H` **(new)**	The calculation itself. Per-cell Goldak evaluation, three guards, renormalisation, self-report	~60 lines
`TEqn.H`	One line: `+ arcSource` on the right-hand side	+1 line
`laserbeamFoam.C`	One line: include the calculation before the energy equation solves	+1 line

Line numbers shift as patches accumulate, so the patcher anchors on text rather than position and refuses if an anchor is missing or ambiguous.

Everything else it touches or reads

PATH	ROLE
`~/thesis/tools/patch_goldak.py`	Applies and reverts all four edits. Writes `.gbak` backups; `--revert` restores them
`~/thesis/cases/caseMatrix.csv`	One row per case. Holds `I_A`, `U_V`, `eta_arc`, the four Goldak lengths and `travel_speed_m_s`
`~/thesis/cases/generate_cases.py`	Derives `arc_power_W` and `arc_x0/y0/z0_m` and writes them into each case
`<case>/constant/caseParameters`	What the solver actually reads at run time. Generated, never hand-edited — regenerating would discard the change

PATH	ROLE
`~/thesis/materials/transportProperties`	Material properties. Separate from the arc, but sets the conductivity and specific heat the deposited power acts on

Version control

Repository	`~/thesis/src/lbf-com` — a fork of the upstream laserbeamFoam clone
Branch	`waam-mhd`, one commit above upstream `OpenFoam_com_main`
Remote	`sabbirdipto/LaserbeamFoam`, pushed. A fork rather than a fresh repo, so the diff renders against upstream and is citable
Built binary	`~/OpenFOAM/sabbirdipto-v2506/platforms/linux64GccDPInt32Opt/bin/laserbeamFoam`

4. How it enters the energy equation

The right-hand side, after the patch:

```
fvScalarMatrix TEqn
(
    fvm::ddt(rhoCp,T) + fvm::div(rhophicp, T)
  - fvm::laplacian(kappa, T)
  ...
  ==
    laser.deposition()      // zero when the laser power table is zero
  + arcSource               // NEW  -- zero when no arc is configured
  + Joulesource             // zero when electromagnetics is off
  + (Qrad_expl - Qv)*gradAlphaDamper
  - TRHS
);
```

Three optional volumetric sources, each collapsing to zero when unused. The pattern is the solver's own — `Joulesource` is built two lines above as `mthd.valid() ? mthd->Joulesource() : zeroJoulesource`. The arc follows it exactly. All three carry units of W/m^3, which is what lets them sit side by side.

5. Reading the parameters

Nothing is hard-coded. Every value comes from `constant/caseParameters`, which `generate_cases.py` writes from a row of `caseMatrix.csv`. Changing a row changes the arc, and the existing `properties_sha` audit already polices comparisons across such changes.

```
IOdictionary arcDict
(
    IOobject
    (
        "caseParameters", runTime.constant(), mesh,
        IOobject::READ_IF_PRESENT, IOobject::NO_WRITE
    )
);

const bool arcOn = arcDict.found("goldak_af_m");

if (arcOn)
{
    arcPower_ = readScalar(arcDict.lookup("arc_power_W"));
    arc_af_   = readScalar(arcDict.lookup("goldak_af_m"));
    ...
    arc_ff_ = 2.0*arc_af_/(arc_af_ + arc_ar_);   // sum to 2
    arc_fr_ = 2.0*arc_ar_/(arc_af_ + arc_ar_);
}
```

`READ_IF_PRESENT` is what makes the patch safe: a case with no such file gets an empty dictionary, `arcOn` is false, and the field stays zero. Every existing tutorial runs unchanged.

6. The calculation

Once per timestep, before the energy equation solves. The two prefactors are computed outside the loop because they do not vary by cell:

```
const scalar xc = arc_x0_ + arcV_*runTime.value();   // torch position

const scalar kf = 6.0*sqrt(3.0)*arc_ff_*arcPower_
                / (arc_af_*arc_b_*arc_c_*M_PI*sqrt(M_PI));
const scalar kr = 6.0*sqrt(3.0)*arc_fr_*arcPower_
                / (arc_ar_*arc_b_*arc_c_*M_PI*sqrt(M_PI));
```

Then one pass over the cells. The branch on the sign of `dx` is what makes it a *double* ellipsoid — the same cell gets a different value depending on which side of the torch it lies:

```
forAll(q, celli)
{
    const scalar dz = Cc[celli].z() - arc_z0_;
    if (dz > 0.0) { q[celli] = 0.0; continue; }        // GUARD 1

    const scalar dx = Cc[celli].x() - xc;
    const scalar dy = Cc[celli].y() - arc_y0_;
    const scalar a  = (dx > 0.0) ? arc_af_ : arc_ar_;  // fore or aft
    const scalar k  = (dx > 0.0) ? kf : kr;

    const scalar e = 3.0*( dx*dx/(a*a)
                         + dy*dy/(arc_b_*arc_b_)
                         + dz*dz/(arc_c_*arc_c_) );
    if (e > 30.0) { q[celli] = 0.0; continue; }        // GUARD 3

    q[celli] = max(min(a1[celli], 1.0), 0.0)           // GUARD 2
             * k * exp(-e);
}
```

Guards 1 and 3 are early exits, so most of the mesh costs two comparisons and nothing more.

7. The three guards, and why each is necessary

GUARD 1 — dz > 0 SKIPPED. The formula is a HALF ellipsoid.

This is the one that is easiest to get wrong and hardest to notice. The 6·sqrt(3) normalisation together with $f_f+f_r=2$ integrates to eta·U·I **only over the half-space below the surface**. Goldak's ellipsoid sits *inside* the workpiece, with c as the penetration depth.

Applied symmetrically about the free surface it delivers **exactly twice** the arc power. Verified numerically before the code was written: symmetric integration gives 3024 W against a 1512 W target; restricted below the surface it gives 1512 W to the digit.

The paper never states this, because their domain made it implicit.

GUARD 2 — GATED ON alpha.metal. The arc must not heat argon.

In a VOF domain part of the ellipsoid sits in shielding gas, which has rho = 1.6 kg/m^3 and c_p = 520 J/kg/K against the metal's 2650 and ~1150. A fraction of 1512 W deposited there would drive it to absurd temperatures — and it would look like a physical result.

GUARD 3 — e > 30 SKIPPED. Cost, not correctness.

exp(−30) is about 9×10^{-14} of the peak, so cells that far out contribute nothing. Skipping them avoids evaluating `exp()` across the whole mesh when the source is confined to a few millimetres around the torch.

On a 768,000-cell domain the ellipsoid occupies a few percent of cells, so this is most of the work avoided.

8. Renormalisation, and the number the solver reports

After the loop, the delivered power is summed and compared with the target. The reduction is parallel-correct, which matters because the arc will usually straddle a processor boundary:

```
scalar raw = 0.0;
forAll(q, celli) { raw += q[celli]*mesh.V()[celli]; }
reduce(raw, sumOp());

const scalar target = arcPower_*arcSym_;   // half domain -> half the arc
if (raw > SMALL)
{
    const scalar scale = target/raw;
    forAll(q, celli) { q[celli] *= scale; }
}
```

The symmetry factor is the third correction. A half domain contains half the arc, so it must receive half the power. Read from the `sym` column; verified numerically at 756 W for the half domain and 1512 W for the full.

Why renormalise at all

Physically the arc delivers eta·U·I to whatever metal is beneath it. If part of the Gaussian falls off the plate — at the start of a track, or over a gap — the real arc still delivers its power to what remains. Renormalising reproduces that, and guarantees the energy budget closes regardless of mesh or free-surface shape.

BUT IT REPORTS WHAT IT HAD TO DO

A correction can hide a mistake. If the ellipsoid were mostly outside the metal, renormalising would silently concentrate the full arc power into whatever few cells remained — producing a violent, plausible-looking result. So the solver prints the raw figure before correction:

```
Arc: 741.2 W in metal below the surface, renormalised x1.0199 to 756 W (raw 98% of target)
```

Raw near 100% is the gate. It means the ellipsoid sits inside the metal and the correction is barely working. Well under 100% in mid-track means something is wrong — and near 50% would point straight at Guard 1 or the symmetry factor.

9. What is verified, and what is not

CLAIM	STATUS	EVIDENCE
Integrates to eta·U·I	**verified**	Numerical integration before coding: 1512.0 W full domain, 756.0 W half, both to 0.00%
Half-ellipsoid convention is required	**verified**	Symmetric integration gives 3024 W — exactly 2x
Compiles and links	**verified**	Zero errors; binary grew 1,971,440 to 1,979,368 bytes
Inert without an arc dictionary	**verified**	Gate A prints `Arc: none` and returns 1.3869e-02, identical to the pre-patch baseline
Produces a correct pool	**NOT TESTED**	No Model P case can start yet — the template lacks its initial conditions, so the arc has never fired
Goldak coefficients suit this rig	**NOT TESTED**	3/6/4/4 mm are Zhao's calibration and must be refitted against a first-layer pool

THE HONEST STATUS

The arc source is written, compiled, regression-tested and committed. It has **never delivered a joule**, because no case exists for it to heat: the Model P template holds a mesh and a control dictionary and nothing else.

Its first real test is the line it prints on step one. Until then the verification above establishes that the arithmetic is right and that it does not disturb anything — not that it produces a correct weld pool.

Next

- Populate the Model P template — initial conditions, `g`, turbulence, laser at zero power, schemes and solvers. Build one case by hand first, then generalise.
- Run a bare plate with no wire. Gate: a pool of roughly the right size, raw power near 100%, and max(T) under about 1600 K — which matters more now that disabling vaporisation removed the temperature ceiling.
- Then arc pressure as `pArc`, following the `pVap` pattern in both `UEqn.H` and `pEqn.H`.
- Recalibrate a_f, a_r, b, c against your own first layer. Zhao's values are a starting guess, not a transferable constant.